고가용성 가상 암호화폐 거래소 운영 설계

한국폴리텍대학 광명융합기술교육원 데이터분석과 파이널 프로젝트 계획서입니다. 1 인 프로젝트입니다.

작성 2026-09-09. 최종 발표 2026-10-16.

이 문서는 저장소 실물에서 다시 유도해 새로 썼습니다. 종전 판(`docs/archive/기획안_원본_20260908.md`)은 폐기했습니다. 그 판은 2026-08-25 부터 여러 결정이 쌓이는 동안 갱신이 따라가지 못해, 저장소와 어긋난 주장이 여러 절에 남아 있었습니다.

이 문서의 수치는 전부 측정 출처를 함께 적었습니다. 측정으로 재현되지 않은 값은 "미측정" 또는 "설계값"으로 표기했습니다.

요약

무엇을 만드는가

봇이 24 시간 매매하는 완전 시뮬레이션 암호화폐 거래소입니다. 실거래와 실자산이 없고, 실행 중에 외부 API 를 부르지 않습니다. 12 개 마켓에서 봇 플릿이 스스로 호가를 만들고 체결을 냅니다.

무엇을 증명하는가

이 프로젝트가 답하려는 질문은 하나입니다.

> **24 시간 도는 거래소에 트래픽이 몰리거나 고장이 나면, 어느 계층이 먼저 터지는가. 그것을 어떻게 알아내고, 무엇을 키워서, 어떻게 안정화하는가.**

핵심은 병목이 한 곳에 고정되어 있지 않다는 점입니다. 한 계층을 키우면 부하가 다음 계층으로 옮겨 갑니다. 데이터베이스를 키우면 요청이 빨리 끝나고, 그만큼 애플리케이션이 더 많은 일을 하게 되어 이번에는 애플리케이션이 병목이 됩니다. 그래서 이 프로젝트는 "한 번 잘 설계한다"가 아니라 "관측하고, 병목을 찾고, 키우고, 옮겨간 병목을 다시 찾는" 반복을 다룹니다.

그 반복을 세 축으로 나눠 증명합니다.

축	증명하려는 것	반증 조건
무결	폭주와 장애 아래에서도 자산별 원장 합이 0 이고, 주문 중복 접수와 중복 체결과 유실이 0 입니다	불변식 27 개 중 어느 하나라도 위반 행이 1 행 나오면 반증입니다
복구	리더 노드를 죽여도 사람 손 없이 다른 노드가 승격해 쓰기가 재개됩니다	30 초를 넘거나, 승격이 일어나지 않거나, 사람이 손대야 재개되면

		반증입니다
안정화	병목 계층을 관측으로 특정하고, 그 계층을 키웠을 때 지표가 예측한 방향으로 움직입니다	병목을 특정하지 못하거나, 키웠는데 지표가 움직이지 않으면 반증입니다

앞의 두 축은 종전 판에서 이어받았습니다. 세 번째 축이 이번에 추가되었습니다. 2026-09-08 실측이 그 축의 첫 구간을 잡았기 때문입니다.

지금까지 무엇이 나왔는가

가장 중요한 실측 셋입니다.

하나. 데이터베이스 규격을 키워도 처리량은 오르지 않습니다. 막고 있는 것이 데이터베이스가 아니기 때문입니다.

데이터베이스 컨테이너의 CPU 한도만 1 코어에서 4 코어로 바꿔 세 쌍을 측정했습니다. 응답 시간 상위 1 퍼센트(p99)가 318 에서 121 밀리초, 382 에서 146 밀리초, 416 에서 127 밀리초로 세 쌍 모두 60 퍼센트 넘게 줄었습니다. 같은 회차에서 초당 체결 수는 16.9 에서 17.7 로 사실상 그대로였습니다. 근거는 `harness/evidence/pool-db-matrix-20260908-204634/`입니다.

다만 그 회차는 부하를 봇으로 걸었고, 봇은 수요가 발생기에 묶입니다. 그래서 규격을 올려도 애플리케이션이 더 할 일이 없었습니다.

2026-09-10 에 개방 루프로 요청 초당 150 을 고정하고 같은 축을 다시 쟀습니다. 네 칸의 제공 부하가 초당 169.5 에서 169.8 로 같았습니다. 코어를 1 에서 8 로 여덟 배 늘리는 동안 초당 접수가 67.7 에서 72.0 으로 6 퍼센트 올랐습니다.

그 회차에서 1 코어 칸이 포화하지 않았습니다. 데이터베이스 CPU 가 72.5 퍼센트인데 수용 거절이 초당 102 였습니다. 하드웨어가 남았는데 요청을 돌려보내고 있었습니다. 커넥션 풀 대기도 네 칸 모두 0 입니다. 막고 있는 것은 수용 정원이고 그 정원은 풀에서 유도되는 설정값이라 코어 수와 무관합니다. 자세한 표는 7.6 에 둡니다.

둘. 커넥션 풀과 수용 정원을 함께 올리면 지연이 오히려 나빠집니다.

풀을 16 에서 48 로 올리면 수용 정원이 6 에서 19 로 따라 오릅니다. 세 쌍 모두 지연이 악화되었습니다(138 에서 223, 86 에서 108, 233 에서 261 밀리초). 커넥션 대기는 전 회차 0 이었으므로 커넥션이 모자란 상태가 아니었고, 정원만 늘리면 데이터베이스 안에서 경합만 커집니다.

셋. 동기 복제는 대기 서버 장애를 그대로 쓰기 정지로 옮깁니다. 그래서 비동기로 내렸습니다.

동기 복제 대상 대기 서버를 한 대만 두고 그 한 대를 죽이면, 주문 접수가 초당 97.2 건에서 2 초 만에 0 건이 됩니다. 두 대를 두고 정족수를 `ANY 1` 로 묶으면 한 대가 죽어도 초당 56 건으로 이어집니다. 다만 같은 표의 마지막 행에서 두 대를 모두 잃으면 다시 0 건입니다. 정족수는 단일 장애점을 이중 장애점으로 옮긴 것이지 없앤 것이 아닙니다. 그래서 2026-09-12 에 동기 정족수를 버리고 대기 서버 한 대 비동기 복제로 내렸습니다. 대가로 RPO 0 주장을 철회합니다 (ADR 0024). 근거는 `harness/evidence/replication-fault-20260908/`입니다.

비용과 일정

발표까지 총계 미국 달러 245.89(원화 344,245 원)입니다. 상한은 달력 월 실청구로 판정하며, 2026 년 9 월과 10 월이 각각 315,954 원으로 자체 상한 500,000 원 안입니다. 이 값은 9.3 표의 행 합에서 나옵니다. 상한은 2026-09-08 에 지도교수 승인으로 100,000 원에서 올렸습니다.

지원 범위 안입니다. 기존 크레딧 잔액 163.71 달러에 더해, 2026-09-08 지도교수 승인분 50 만원 이상이 다음 주에 크레딧으로 지급될 예정입니다. 종전 판은 총계를 157.91 달러로 적고 잔여 5.80 달러가 남는다고 했는데, 그 계산이 확정 단가 소계를 153.66 이 아니라 135.77 로 잘못 쓴 데서 나왔습니다.

일정은 2026-09-01 착수, 10-09 기능 동결, 10-16 발표입니다. 09-08 로 잡았던 외부 접속 성립 고정점은 달성하지 못했습니다. 클라우드에 인스턴스를 세운 적이 0 건이고 도메인 위임도 끝나지 않았습니다.

1. 프로젝트 개요

1.1 한 줄 정의

봇 플릿이 실시간으로 매매하는 완전 시뮬레이션 암호화폐 거래소입니다. 실거래와 실자산과 런타임 외부 API 의존이 없습니다. 이 시스템에 장애를 주입하면서 무결과 복구와 안정화를 측정합니다.

"완전 시뮬레이션"이라는 말은 시장 데이터까지 내부에서 만든다는 뜻입니다. 실행 중에 외부 거래소 API 를 부르지 않습니다. 그래야 외부 서비스의 장애가 우리 실험 결과에 섞이지 않습니다.

1.2 왜 거래소인가

거래소를 고른 것은 개발 취향이 아니라 실험 설계 판단입니다.

첫째, 복식부기 원장이 회계 불변식을 도메인 성질로 제공합니다. 복식부기란 모든 거래를 차변과 대변 두 쪽에 같은 금액으로 적는 방식입니다. 그래서 자산별로 모든 항목을 더하면 항상 0 이어야 합니다. 이 0 이라는 숫자가 판정 술어가 됩니다. 사람이 눈으로 보고 "정상인 것 같다"고 말하는 대신, SQL 한 줄이 위반 행 수를 돌려줍니다.

둘째, 주문 처리는 순서가 결과를 바꿉니다. 같은 주문 묶음도 처리 순서가 다르면 체결 결과가 달라집니다. 그래서 장애 뒤에 상태를 복원할 때 "순서까지 같게 복원했는가"를 물을 수 있습니다.

셋째, 24 시간 도는 시스템이라 정지 자체가 사고입니다. 야간에 사람이 없어도 복구되어야 한다는 요구가 자연스럽게 나옵니다.

1.3 무엇을 증명하는가

기능 목록이 아니라 명제를 증명합니다. 명제는 반증 조건을 함께 적을 때만 명제입니다.

구분	증명 1 체결 무결	증명 2 자동 복구	증명 3 병목 이동과 안정화
주장	폭주와 장애 아래에서도	리더를 죽여도 사람 손 없이 승격해	병목 계층을 관측으로

	자산별 원장 합이 0 이고 중복과 유실이 0 입니다	쓰기가 재개됩니다	특정하고, 키웠을 때 지표가 예측대로 움직입니다
관측 대상	`ledger_entries`, `balances`, `command_journal`, `event_journal`, `trades`	`leader_lease.epoch`, `command_journal.fencing_token`, `trades.engine_ts`	계층별 CPU, 응답 시간 분위수, 수용 정원 점유, 커넥션 대기
판정 술어	불변식 27 개의 위반 행 수	승격 후 최초 신규 체결의 엔진 시각에서 정지 시각을 뺀 값	조작 전후 지표 차이와 그 방향
통과 기준	전 항목 0 행	30 초 이내, 세대 번호 정확히 1 증가, 수동 개입 0 회	예측한 방향으로 유의하게 움직이고 재현됩니다
반증 조건	어느 술어라도 1 행 반환	30 초 초과, 승격 미발생, 사람 개입 필요	병목 미특정, 또는 조작이 지표를 움직이지 못함

세 번째 축의 "예측한 방향"이란 이런 뜻입니다. 데이터베이스가 병목이라고 판정했으면, 데이터베이스를 키웠을 때 응답 시간이 줄어야 합니다. 줄지 않으면 병목 판정이 틀린 것입니다.

1.4 무엇을 하지 않는가

- 실제 화폐와 실제 자산을 다루지 않습니다.
- 실행 중에 외부 거래소 API 를 부르지 않습니다. 초기 시세 데이터는 빌드 시점에 고정한 값입니다.
- 사람이 눈으로 보고 판정하지 않습니다. 모든 판정은 SQL 위반 행 수, 지표 임계, 종료 코드 중 하나입니다.

2. 문제 정의

2.1 24 시간 거래소에서 무엇이 터지는가

거래소는 멈추면 안 되는 시스템입니다. 그런데 멈추게 만드는 원인은 하나가 아닙니다. 이 프로젝트는 원인을 세 갈래로 나눕니다.

갈래 하나. 트래픽이 몰릴 때.

주문이 평소보다 훨씬 많이 들어오면 어딘가가 먼저 한계에 닿습니다. 그 "어딘가"가 어디인지는 미리 알 수 없습니다. 데이터베이스일 수도 있고, 애플리케이션의 CPU 일 수도 있고, 커넥션 풀일 수도 있고, 수용 정원일 수도 있습니다. 관측으로 특정해야 합니다.

갈래 둘. 부품이 고장 날 때.

노드 하나가 죽거나, 네트워크가 끊기거나, 디스크가 느려집니다. 이때 시스템 전체가 멈추는지, 아니면 남은 부품으로 계속 도는지가 갈립니다. 그리고 고장이 끝난 뒤 스스로 돌아오는지도 갈립니다.

갈래 셋. 고장이 아닌데 느려질 때.

프로세스가 죽지 않고 살아 있는데 응답만 안 하는 상태가 있습니다. 이것을 회색 장애라고 부릅니다. 죽은 것보다 다루기 어렵습니다. 감시 장치가 "살아 있음"만 보면 정상으로 판정하기 때문입니다.

2.2 왜 병목이 이동하는가

한 계층을 키우면 그 계층이 더 이상 병목이 아니게 됩니다. 그런데 그 순간 부하가 다음 계층으로 옮겨 갑니다.

구체적으로 이렇습니다. 데이터베이스가 느리면 애플리케이션은 데이터베이스 응답을 기다리며 놉니다. 이때 애플리케이션 CPU 는 낮습니다. 데이터베이스를 키우면 응답이 빨라지고, 애플리케이션은 기다리는 시간이 줄어 더 많은 요청을 처리합니다. 그러면 이번에는 애플리케이션 CPU 가 올라갑니다.

그래서 "데이터베이스를 키웠더니 애플리케이션이 터졌다"는 일이 실제로 일어납니다. 그것은 실패가 아니라 병목이 이동한 것입니다. 다음 조작은 애플리케이션 노드를 늘리는 것입니다.

이 이동을 눈으로 보려면 두 가지가 필요합니다. 계층별 지표를 동시에 보는 관측과, 한 번에 한 축만 바꾸는 실험 설계입니다.

2.3 왜 이것을 증명이라고 부르는가

"고가용성 시스템을 만들었습니다"라는 문장은 검증할 수 없습니다. 무엇이 참이면 그 문장이 거짓이 되는지가 없기 때문입니다.

그래서 이 프로젝트는 모든 주장에 반증 조건을 붙입니다. 원장 합이 0 이 아니면 무결이 깨진 것입니다. 30 초 안에 복구되지 않으면 자동 복구가 깨진 것입니다. 데이터베이스를 키웠는데 응답 시간이 안 줄면 병목 판정이 틀린 것입니다.

판정은 사람이 하지 않습니다. SQL 이 위반 행 수를 돌려주고, 스크립트가 종료 코드를 돌려줍니다.

3. 현재 진척

3.1 성립한 것

아래는 저장소에 실물이 있고 측정으로 확인한 항목입니다.

축	상태	근거
매칭엔진	성립. 단일 라이터 구조, 결정론적 재생	`server/app/src/main/java/kr/exchange/engine/`
복식부기 원장	성립. 불변식 27 개를 SQL 로 판정	`harness/sql/invariants.sql`
리더 선출과 세대 번호 펜싱	성립. 세대 번호가 엄격히 증가하고 구 리더의 쓰기가	`harness/evidence/leader-failover-20260908/`

	거부됨	
리더 임차를 커밋 경로 밖으로 분리	성립. 다만 선택 경로이고 기본값이 아님	ADR 0017, `harness/evidence/leader-lease-dynamodb-20260908/`, `harness/evidence/load-ceiling-20260909/`, `harness/evidence/k6-ceiling-20260909/`
부하 중 노드 재합류	성립. 재합류 20.8 초, 그 창의 접수가 정상 구간과 같음	`harness/evidence/follower-rejoin-20260908/`
비동기 2 노드 복제	성립. 대기 서버 상실이 primary 커밋을 멈추지 않음. 로컬 한 호스트 두 컨테이너 확인	ADR 0024, `deploy/compose.node-data.yml`
수용 제한	성립. 커넥션 풀에서 정원을 유도하고 초과를 거절로 막음	`server/app/src/main/java/kr/exchange/api/common/AdmissionLimits.java`
카오스 주입	성립. 스크립트 9 종	`harness/scripts/` 의 e1 부터 e9
관측 자산	성립. 레코딩 규칙 9 개, 알람 17 개, 대시보드 9 종	`harness/prometheus/rules/`, `harness/grafana/dashboards/`
통합 검증	성립. 검증 클래스 74 개가 전량 통과	`server/app/src/test/`

3.2 성립하지 않은 것

정직하게 적습니다. 문서에 설계가 있다는 이유로 완료로 적지 않습니다.

축	상태	무엇이 없는가
클라우드 배포	미성립	AWS 에 배포한 적이 없습니다. `.tf` 파일이 0 개입니다
상시면 관측 배선	미성립	배포용 컴포즈 3 종에 관측 서비스 정의가 0 건입니다. 관측 스택은 로컬 카오스 스택에서만 돕니다
계층별 프로브	미성립	네트워크 계층과 전송 계층과 응용 계층을 각각 찌르는 프로브가 배선되지 않았습니다
알림 실투입	미성립	알림 수신 주소를 넣지 않았습니다. 로컬에서 도달만 확인했습니다
커널 계층 장애 주입	미성립	실험 E10 이 미실시입니다. 종전 E1 부터

		E9 까지는 전부 프록시 모사입니다
데이터베이스 자동 페일오버	미성립	`pg_ctl promote`, `pg_promote()`, `promote_trigger_file` 이 저장소 전체에 0 건입니다. 수동 절차만 있고 `docs/operations/DB_수동_페일오버.md` 에 있습니다 (ADR 0024)
데이터베이스 페일오버 리허설	부분	승격 명령과 재편입은 단일 호스트에서 실측했습니다 (`harness/evidence/failover-rpo-20260912/`). 장애를 알아차리는 시간, 승격 여부를 판단하는 시간, 앱의 접속 대상을 바꾸는 시간은 재지 않았습니다. 절차 전체를 다중 호스트에서 돌린 적이 0 건입니다

3.3 측정 평면의 한계

지금까지의 실측은 전부 로컬 도커 환경입니다. 배포 대상 인스턴스 실물에서 잰 값이 0 건입니다.

그리고 같은 기계에서 여러 컨테이너가 함께 돌았습니다. 2026-09-08 회차에서는 스택 12 개가 상시 돌았고, 통합 검증이 겹친 구간에는 임시 데이터베이스 컨테이너가 최대 27 개까지 함께 있었습니다. 그래서 이 문서의 성능 수치는 절대값이 아니라 같은 주행 안의 상대 비교로 읽어야 합니다.

4. 기술 선택

4.1 선택의 원칙

기술을 고를 때 기준이 하나입니다. **장애를 주입할 권한이 없는 계층은 증명 대상에서 빠집니다.**

이 프로젝트가 증명하려는 것이 장애 아래의 무결과 복구인데, 그 장애를 만들 수 없으면 증명할 것이 없습니다. 그래서 편한 관리형 서비스 대신 직접 운영하는 쪽을 여러 번 골랐습니다. 대가는 정직하게 계상합니다.

4.2 선택표

축	선택	사유
언어와 프레임워크	Java 21, Spring Boot	가상 스레드로 봇 수천 개를 한 프로세스에서 돌립니다
데이터베이스	EC2 에 직접 운영하는 PostgreSQL 17	관리형 데이터베이스는 운영체제 접근이 없어 커널 파라미터 조작과 프로세스 정지를 못 합니다. 그러면 데이터베이스 계층이 증명 범위에서 빠집니다 (ADR 0004)

컴퓨트	EC2 Graviton (t4g, c7g)	운영체제 셸 접근이 필요합니다. Graviton 은 x86 대비 상시 구동에서 약 20 퍼센트 저렴합니다
로드밸런서	Application Load Balancer, 2 가용영역	두 가용영역의 진입점을 이중화하려면 관리형이 필요합니다. 헬스체크가 애플리케이션의 상태 확인 경로를 그대로 읽으므로 판정 주체가 애플리케이션이 됩니다
확장	EC2 Auto Scaling 그룹. 확장 정책은 두지 않음	조회 계층에만 겁니다. 최소 2, 희망 2, 최대 3 입니다. 확장 방식은 수동입니다. 자동 확장만 관리형 감시 서비스를 요구하는데 그 서비스를 비용으로 기각했으므로, 정책 없이 희망 용량만 직접 설정합니다 (ADR 0015)
리더 임차	임차 보관소를 갈아 끼울 수 있게 두고 구현을 둘 둡니다. 데이터베이스 행과 DynamoDB 항목 1 개입니다	임차 갱신이 데이터베이스 커밋이면, primary 가 멈출 때 갱신도 함께 멈춥니다. 그러면 임차가 만료되고 복제가 돌아와도 애플리케이션 재기동 없이는 복구되지 않습니다. DynamoDB 구현이 그 결합을 끊습니다. 기본값은 데이터베이스 쪽으로 두었습니다. 옮기기 전과 옮긴 뒤를 같은 조건에서 재려면 종전 경로가 살아 있어야 하기 때문입니다 (ADR 0017)
관측	Prometheus 와 Grafana 직접 운영	레코딩 규칙 9 개와 알람 17 개와 대시보드 9 종이 이미 파일로 있고 지속적 통합이 검증합니다. 관리형 감시 서비스로 옮기면 월 약 190 달러가 듭니다
네트워크 주소 변환	관리형 게이트웨이 2 개, 가용영역별로 하나씩	증명 축 셋 어디에도 들어가지 않는 배관입니다. 4.1 원칙에 따라 장애 주입 대상이 아니므로 관리형을 씁니다. 인스턴스로 직접 운영하면 그 노드 자체가 가용영역별 단일 장애점이 되어, 증명 대상이 아닌 부품의 고장이 실험 결과에 섞입니다 (ADR 0020)
정적 웹	S3 와 CloudFront	애플리케이션 노드 자원을 쓰지 않고, 원본 장애와 웹 가용성을 분리합니다
시크릿	SSM 파라미터 스토어	시크릿당 월 0.40 달러를 아낍니다
관리자 접근	SSM 세션 매니저	원격 접속 포트를 아예 열지 않습니다. 세션 기록이 그 자체로 감사 증거입니다
지속적 통합 인증	GitHub Actions OIDC 페더레이션	장기 접근 키를 발급하지 않습니다

오케스트레이터 (클라우드)	채택하지 않음	스케줄러가 컨테이너를 자동 재기동하면 노드를 죽이는 실험의 관측이 오염됩니다
인프라 코드화	Terraform. 여유 시간 조건부	아직 착수하지 않았습니다

4.3 비채택과 그 사유

비채택	사유
관리형 관계형 데이터베이스	운영체제 접근이 없어 데이터베이스 계층 장애 주입이 불가능합니다
관리형 감시 서비스	월 약 190 달러입니다. 그리고 자동 확장 정책이 이 서비스를 요구하므로, 비채택이 곧 수동 확장 채택으로 이어집니다
관리형 쿠버네티스	컨트롤플레인 자체가 시간 과금입니다
상시 사설망 터널	두 평면을 잇는 것은 네트워크가 아니라 같은 이미지 지문과 저장소에 커밋한 판정 파일입니다
시크릿 매니저	표준 파라미터로 충분합니다

5. 평면 분리

5.1 왜 평면을 나누는가

학교 필수 조건인 외부 상시 접속과, 이 프로젝트가 필요로 하는 커널 계층 장애 주입은 한 평면에서 양립하지 않습니다. 커널에 장애를 주입하면 그 노드에 접근하는 경로도 함께 끊길 수 있기 때문입니다.

그래서 평면을 셋으로 나눕니다.

평면	위치	구동	허용하는 실험
상시면	AWS 서울 리전	24 시간	애플리케이션 계층 정지와 가용영역 상실 재현. 커널 주입은 제외합니다
증명면	AWS, 창 구동	실험과 발표 창	부하 주입과 확정 회차 재현. 창이 끝나면 볼륨까지 파기합니다
검증면	온프레미스, 세션 구동	실험 세션 중	커널 계층 주입 전량. 비용이 0 이라 회차 수에 제약이 없습니다

검증면의 실물 구성은 `제출_20260901/diagram/검증면_도커구성.png` 에 있습니다. 컨테이너 배치와 함께 무엇이 시스템이고 무엇이 계측기인지, 장애를 어디에 꽂는지, 관측이 어느 쪽에 서 있는지를 표시했습니다. 실험 증거가 전부 이 평면에서 나왔으므로 그 출처를 그림으로 남깁니다. 건수는 10.4 가 갖습니다. 이 자리에 복사하지 않습니다.

5.2 배치 규칙

> 상시 필요한 것은 AWS 에 둡니다. 세션 중에만 필요한 것은 검증면에 둡니다.

이 규칙이 관측 배치를 정합니다. Prometheus 는 대상에게 가서 지표를 가져오는 방식입니다. 가져오기를 놓치면 그 구간의 시계열이 영구 공백이 됩니다. 나중에 채울 수단이 없습니다. 그래서 판정에 걸린 관측은 대상과 같은 평면인 AWS 에 둡니다.

5.3 두 평면을 잇는 것

상시 터널을 두지 않습니다. 두 평면을 잇는 것은 네트워크가 아니라 둘입니다.

- 같은 컨테이너 이미지 지문. 지속적 통합이 이미지를 만들어 레지스트리에 올리고, 양쪽이 같은 지문을 받습니다.
- 저장소에 커밋한 판정 파일. 검증면에서 나온 실험 결과를 저장소에 넣고, 그것이 증거가 됩니다.

검증면 기계가 상시 구동이 아니므로 상시 터널을 전제하면 그 전제가 먼저 깨집니다.

6. 인프라와 배포

6.1 전체 구조

사용자 요청은 도메인 이름 해석을 거쳐 콘텐츠 전송 네트워크로 들어옵니다. 정적 화면은 오브젝트 스토리지에서 바로 나가고, API 와 실시간 연결은 로드밸런서로 넘어갑니다.

로드밸런서 뒤에는 애플리케이션 노드가 있고, 그 뒤에 데이터베이스가 있습니다. 리더 임차는 별도 저장소로 뺄 수 있게 해 두었습니다.

6.2 노드 구성

노드	등급	가용영역	구동	역할
app-1	t4g.small	a	상시	리더. 매칭엔진이 여기서만 돕니다
app-2	t4g.small	c	상시	조회를 받습니다. 리더가 죽으면 승격 대상입니다
app-3	t4g.small	c	증설분	평시에는 없습니다. 관리자가 버튼으로 늘립니다
pg-primary	t4g.small	a	상시	쓰기를 받습니다
pg-standby	t4g.small	c	상시	비동기 스트리밍 복제를 받습니다. hot standby 로 띄우되

				조회 라우팅은 구현하지 않았습니다 (ADR 0024)

(행 삭제)

| bot-1 | t4g.small | c | 상시 | 봇 플릿과 관측 스택이 함께 돕니다 |

| load | c7g.large | c | 실험 창 | 부하 발생기 전용입니다. 버스트가 없는 등급이라 신용 소진이 측정에 섞이지 않습니다 |

사설 서브넷의 외부 접속 경로는 관리형 주소 변환 게이트웨이입니다. 가용영역마다 하나씩 두 개입니다. 노드가 아니라 관리형 자원이므로 이 표에 없습니다 (ADR 0020).

봇과 관측을 같은 노드에 둔 이유는 둘입니다. 감시는 피검체 밖에 있어야 성립하고, 가용영역 a 를 잃었을 때 승격을 관측할 수 있어야 합니다 (ADR 0016).

6.3 비동기 복제와 수동 승격

데이터베이스는 primary 한 대와 대기 서버 한 대입니다. 비동기 스트리밍 복제이고 `synchronous_standby_names` 를 비워 둡니다. 커밋할 때 대기 서버의 응답을 기다리지 않는다는 뜻입니다. `synchronous_commit` 은 on 을 유지하므로 primary 자신의 미리 쓰기 기록 저장까지는 기다립니다 (ADR 0024).

비동기를 고른 이유는 가용성입니다. 동기 복제는 대기 서버 장애를 그대로 쓰기 정지로 옮깁니다. 2026-09-08 실측이 그것입니다.

조건	초당 접수	서버 오류	커밋
동기 대기 1 대 정상	97.2	0	정상
동기 대기 1 대 정지	0.0	0	전면 정지
정족수, 2 대 중 1 대 정지	56.0	0	정상 지속
정족수, 2 대 모두 정지	0.0	0	전면 정지

배치는 primary 가 가용영역 a, 대기 서버가 가용영역 c 입니다. 정족수가 없으므로 가용영역 c 를 잃어도 primary 의 커밋은 계속됩니다. 가용영역 a 를 잃으면 쓰기가 멈추고, 복구는 대기 서버를 수동으로 승격하는 것입니다. 절차는 `docs/operations/DB_수동_페일오버.md` 에 있습니다.

2026-09-12 에 그 절차의 일부를 쟀습니다 (`harness/evidence/failover-rpo-20260912/`).

항목	값
승격 명령 실행	1.697 초, 1.886 초
유실, 복제가 따라오는 중 급사	응답을 받은 88 건 중 0 건
유실, 대기 서버를 8 초 멈춘 뒤 급사	응답을 받은 122 건 중 36 건
구 primary 재편입, 전량 복사 428.7MiB	264.7 초. 복제 재개 확인까지 합 272.5 초

승격 명령이 1.7 초대라는 것을 복구 시간으로 읽지 않습니다. 사람이 장애를 알아차리는 시간, 승격할지 판단하는 시간, 앱의 접속 대상을 바꾸는 시간이 그 안에 없고 그 셋이 실제 복구 시간을 지배합니다. 유실량은 그 순간의 복제 지연과 같습니다. 정상 구간의 0 건을 무손실 보장으로 읽지 않습니다 (ADR 0024).

6.4 조회 라우팅

로드밸런서 뒤에 대상 그룹이 둘입니다.

대상 그룹	대상	트래픽 포트	헬스체크 경로
쓰기	리더 1 대	8080	준비 상태 경로
조회	리더와 조회 노드 전부. 2 대에서 3 대 가변	8080	읽기 전용 상태 경로

두 대상 그룹의 트래픽 포트는 둘 다 8080 이고 헬스체크 포트도 둘 다 8081 입니다. 갈리는 것은 헬스체크 경로입니다.

조회 대상에 리더가 들어갑니다. 리더를 빼면 조회 분산이 구조적으로 0 이 되기 때문입니다. 팔로워는 준비 상태 경로에서 거절을 내므로 쓰기 대상 그룹에서만 빠집니다.

6.5 리더 임차와 세대 번호

한 시점에 쓰기를 할 수 있는 노드는 하나입니다. 그 권한을 임차로 관리합니다.

임차 갱신은 조건부 쓰기로 합니다. 초당 한 번입니다.

보관소는 갈아 끼울 수 있습니다. 구현이 둘입니다. 데이터베이스 행을 쓰는 것과 DynamoDB 항목 하나를 쓰는 것입니다. 설정 한 줄로 고릅니다.

현재 기본값은 데이터베이스 쪽입니다. 옮기기 전과 옮긴 뒤를 같은 조건에서 재려면 종전 경로가 살아 있어야 하기 때문입니다. DynamoDB 쪽을 고르면 갱신이 데이터베이스 커밋 경로 밖으로 나갑니다. 아래에 적은 실측이 그 경로의 것입니다 (ADR 0017).

세대 번호 검증은 데이터베이스에 남습니다. 쓰기 트랜잭션 안에서 "지금 이 노드가 현재 세대의 리더인가"를 확인합니다. 구 리더가 살아 돌아와 쓰려고 하면 그 검증에서 막힙니다.

임차 갱신을 데이터베이스 밖으로 옮긴 이유는 실측에서 나왔습니다. 갱신 자체가 커밋이면 커밋이 막힐 때 갱신도 함께 멈춥니다. 그 실측은 동기 구성에서 대기 서버를 전부 잃어 만든 것이고, 비동기로 내린 지금은 같은 연쇄의 진입 조건이 primary 자체의 정지로 좁아졌습니다 (ADR 0024). 그러면 임차가 만료되는데 그것을 확인할 커밋이 막혀 있어 아무도 모르고, 복제가 돌아와 커밋이 풀리는 순간 세대 번호 검증이 리더를 끌어내립니다. 옮긴 뒤 같은 장애를 다시 내니 87 초 정지 동안 갱신이 초당 0.35 회로 계속되었고 애플리케이션 재기동 없이 3 초 만에 정상화되었습니다 (ADR 0017).

6.6 웹 배치

웹은 빌드 시점에 정적 파일로 굽습니다. 실행 시점에 서버가 필요 없습니다.

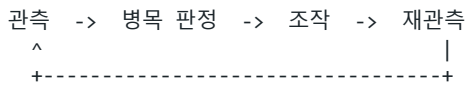
API 주소는 빈 문자열입니다. 같은 출처로 부른다는 뜻입니다. 콘텐츠 전송 네트워크가 경로로 갈라, 정적 파일은 오브젝트 스토리지에서 API 는 로드밸런서에서 가져옵니다. 그래서 브라우저 입장에서는 전부 같은 주소이고 교차 출처 문제가 없습니다.

7. 운영. 관측에서 안정화까지

이 절이 세 번째 증명 축을 담습니다. 부하가 몰리거나 고장이 났을 때 무엇을 보고 무엇을 하는지를 순서로 적습니다.

7.1 순환 구조

운영은 네 단계의 반복입니다.



한 바퀴가 끝나면 병목이 다른 계층으로 옮겨가 있습니다. 그래서 같은 절차를 다시 돕니다. 이 반복이 안정화입니다.

7.2 무엇을 보는가

계층별로 지표를 나눠 봅니다. 한 계층만 보면 병목을 특정할 수 없습니다.

계층	지표	이 지표가 높으면
진입	로드밸런서의 노드별 응답 시간과 오류율	특정 노드만 나쁘면 그 노드 문제, 전부 나쁘면 하류 문제입니다
애플리케이션	프로세스 CPU, 응답 시간 분위수, 수용 정원 점유	CPU 가 높으면 노드를 늘립니다
수용 제한	거절 수, 정원 점유율	정원이 꽉 차 거절이 나면 상류에서 막고 있는 것입니다
커넥션	사용 중 수, 대기 수, 확보 시간	대기가 있으면 커넥션이 모자란 것입니다
데이터베이스	CPU, 커밋 지연, 복제 지연, 잠금	CPU 가 한도에 붙으면 규격을 올립니다

관측 자산은 레코딩 규칙 9 개, 알람 17 개, 대시보드 9 종입니다. 지속적 통합이 규칙 문법을 검사합니다.

7.3 어느 계층이 병목인지 어떻게 아는가

지표 하나만 보면 오판합니다. 조합으로 봅니다.

관측	병목 판정
데이터베이스 CPU 가 한도에 붙고, 애플리케이션 CPU 는	데이터베이스가 병목입니다

낮고, 커넥션 대기가 있다	
애플리케이션 CPU 가 높고, 데이터베이스 CPU 는 여유가 있다	애플리케이션이 병목입니다. 노드를 늘립니다
양쪽 CPU 가 다 낮은데 거절이 난다	수용 정원이 병목입니다
양쪽 CPU 가 다 낮고 커넥션 대기도 없는데 느리다	저장소 지연일 가능성이 큽니다
커넥션 대기가 있는데 데이터베이스 CPU 가 낮다	커넥션 풀 크기가 병목입니다

2026-09-08 실측에서 마지막 두 줄을 구분하는 값이 나왔습니다. 커넥션 대기가 전 회차 0 이었고 데이터베이스가 자기 한도의 절반도 안 쓰고 있었습니다. 데이터베이스 대기 사유를 열어 보니 38 개 커넥션 중 35 개가 "클라이언트의 다음 요청을 기다리는 중"이었습니다. 밀려서 막힌 것이 아니라 일이 없어서 노는 상태였습니다.

7.4 무엇을 조작하는가

계층마다 조작 수단이 다릅니다.

계층	조작	방식	효과
조회 애플리케이션	노드 수 증감	관리 화면 버튼	조회 용량이 늘어납니다
쓰기 애플리케이션	없음		단일 라이터라 노드를 늘려도 쓰기 천장이 안 오릅니다
데이터베이스	규격 상향	인스턴스 교체	응답 시간이 줄어듭니다
커넥션 풀과 정원	크기 조정	설정	실측에서는 지연이 악화되었습니다

조회 노드는 버튼이고 데이터베이스는 권고입니다. 조회 노드는 늘리면 바로 용량이 늘어 버튼으로 둘 값이 있습니다. 데이터베이스는 늘려도 쓰기 천장이 안 오르고 규격 상향은 인스턴스 교체라 버튼으로 누를 일이 아닙니다.

7.5 스케일링 실행 주체

판정은 자동이고 실행은 사람입니다 (ADR 0015).

Prometheus 가 노드 지표를 모으고 임계를 넘으면 관리 화면에 "부하 높음, 증설 권장"을 표시합니다. 관리자가 버튼을 누르면 요청이 관리 API 를 지나 클라우드 API 를 부르고 조회 노드가 한 대 늡니다.

실행을 사람에게 둔 이유가 셋입니다.

첫째, 스케줄러가 컨테이너를 자동 재기동하면 노드를 죽이는 실험의 관측이 오염됩니다. 복구 시작 시점을 사람이 정하면 관측 창을 사람이 소유합니다.

둘째, CPU 하나로는 부하와 정지를 구분하지 못합니다. 정지한 노드의 CPU 를 실측했더니 1.00 에서 1.05 코어였고, 이는 임계 근처입니다.

셋째, 증설이 답이 아닌 축이 있습니다. 주문은 단일 라이터라 노드를 늘려도 오르지 않습니다. 어느 축이 포화인지 가르는 것은 판별 절차이고 확장기가 대신하지 못합니다.

7.6 데이터베이스 규격 상향

데이터베이스는 노드 수가 아니라 규격을 바꿉니다. 확장 대상이 조회 계층뿐이라는 규정은 노드 수 증감과 그 버튼화를 배제한 것이지 규격 상향까지 배제한 것이 아닙니다.

2026-09-08 실측에서 규격 상향의 값이 나왔습니다. 데이터베이스 CPU 한도만 1 코어에서 4 코어로 바꿨습니다.

조건	응답 시간 상위 5 퍼센트	상위 1 퍼센트
풀 16, 40 OPS	138 에서 86 밀리초	318 에서 121 밀리초
풀 48, 40 OPS	223 에서 108 밀리초	382 에서 146 밀리초
풀 16, 고부하 회차	233 에서 96 밀리초	416 에서 127 밀리초

세 쌍 모두 상위 1 퍼센트가 60 퍼센트 넘게 줄었습니다. 꼬리가 먼저 줄어듭니다.

같은 회차에서 데이터베이스가 쓰는 CPU 양 자체는 줄지 않았습니다. 방향이 일관되지 않고 크기도 표본 산포 안이었습니다. 코어를 늘리는 것은 같은 일을 넓게 펴는 것이지 일을 줄이는 것이 아닙니다. 얻는 것은 대기 시간 제거입니다.

초당 체결 수는 16.9 에서 17.7 로 사실상 그대로였습니다. 수요를 천장 아래로 고정했기 때문입니다. 요청이 몰릴 때는 다릅니다. 요청 하나가 정원 한 칸을 붙들고 있는 시간이 3 분의 1 로 줄면 같은 정원으로 더 많이 통과시킬 수 있습니다.

종전 회차가 부하를 봇으로 걸어 수요가 발생기에 묶여 있었기 때문에 규격을 올려도 더 할 일이 없었습니다. 개방 루프로 다시 잰 값은 아래에 둡니다.

데이터베이스 규격	초당 접수	초당 체결	응답 시간 상위 1 퍼센트
1 코어 1 기가	10.3	8.9	3,747 밀리초
2 코어 2 기가	13.5	14.1	645 밀리초
4 코어 4 기가	17.8	19.0	468 밀리초
8 코어 8 기가	17.7	17.8	354 밀리초

1 코어에서 4 코어로 가면 접수가 1.7 배, 체결이 2.1 배, 상위 1 퍼센트가 8 배 좋아집니다. 규격이 정산과 발행 구간의 데이터베이스 왕복을 줄이고 그것이 처리량으로 나옵니다.

이 표는 봇으로 부하를 건 회차의 값입니다. 봇은 수요가 발생기에 묶이므로 규격을 올려도 더 할 일이 없습니다. 곧 이 표의 상승분은 규격의 효과가 아니라 봇 수요의 한계를 본 것입니다.

2026-09-10 에 개방 루프로 요청 초당 150 을 고정하고 같은 축을 다시 쟀습니다. 네 칸의 제공 부하가 초당 169.5 에서 169.8 로 같았습니다.

데이터베이스 규격	초당 접수	초당 체결	초당 수용 거절	상위 1 퍼센트	데이터베이스 CPU
1 코어 1 기가	67.7	10.1	102.0	491 밀리초	72.5 퍼센트
2 코어 2 기가	68.7	9.3	101.0	602 밀리초	93.8 퍼센트
4 코어 4 기가	71.0	10.6	98.6	475 밀리초	61.1 퍼센트

8 코어 8 기가	72.0	8.9	97.8	466 밀리초	70.4 퍼센트

코어를 여덟 배 늘리는 동안 접수가 6 퍼센트 올랐습니다. 다만 이 판정은 2026-09-10 마켓별 샤딩으로 뒤집혔습니다. 같은 축을 12 마켓에서 다시 재니 1 코어 44.0 에서 4 코어 105.8 로 2.4 배 올랐고 1 코어 데이터베이스 CPU 가 99.0 으로 진짜 포화했습니다. 규격 상향이 무의미했던 것은 규격의 성질이 아니라 코드가 자원을 못 쓰는 성질이었습니다(ADR 0021, `제출_20260901/테스트_정본.md` 5.7.6).

1 코어 칸이 포화하지 않았습니다. 데이터베이스 CPU 가 72.5 퍼센트인데 수용 거절이 초당 102 입니다. 하드웨어가 남았는데 요청을 돌려보내고 있습니다. 커넥션 풀 대기도 네 칸 모두 0 입니다. 막고 있는 것은 데이터베이스가 아니라 수용 정원이고, 그 정원은 풀에서 유도되는 설정값이라 코어 수와 무관합니다.

따라서 이 시스템에서 데이터베이스 규격 상향은 처리량 대책이 아닙니다. 처리량을 올리려면 정원을 올리거나 그 위의 직렬 구간을 줄여야 합니다.

두 회차의 값과 무효 사유는 `제출_20260901/테스트_정본.md` 3.2 와 5.2 에 둡니다. 포화 지점을 인용해야 하면 그 문서를 보십시오. 이 표만으로 4 코어 포화를 단정하지 마십시오.

7.7 무중단 규격 상향 절차

규격 상향을 서비스 중단 없이 하는 절차가 비동기 복제 위에 섭니다. 대기 서버를 내려도 primary 커밋이 멈추지 않으므로 대기 서버가 한 대여도 이 절차가 성립합니다. 동기 구성이었다면 대기 서버를 내리는 순간 커밋이 전면 정지해 절차 자체가 불가능했습니다 (ADR 0024).

단계	조작	그동안의 쓰기
1	대기 서버를 큰 규격으로 교체	계속됩니다. 비동기라 대기 서버 부재가 커밋을 막지 않습니다
2	복제가 따라잡을 때까지 대기	계속됩니다
3	대기 서버를 수동 승격	승격 창에서 멈춥니다
4	옛 primary 를 큰 규격으로 교체해 대기 서버로 재합류	계속됩니다

멈추는 구간은 3 단계 하나뿐이고 그 길이는 승격 시간입니다. 승격은 자동이 아니라 사람이 하는 절차입니다. 승격 명령 자체는 1.7 초대로 끝나지만 (6.3), 장애를 알아차리고 승격할지 판단하고 앱의 접속 대상을 바꾸는 시간이 그 앞뒤에 붙고 그 구간은 미측정입니다. 4 단계 재합류도 이 구조에서는 전량 복사라 2026-09-12 회차에서 264.7 초가 걸렸습니다. 절차는 `docs/operations/DB_수동_페일오버.md` 에 있습니다 (ADR 0024).

7.8 병목이 옮겨간 뒤

데이터베이스를 키우면 응답이 빨라지고 애플리케이션이 더 많은 요청을 처리합니다. 그러면 애플리케이션 CPU 가 올라갑니다. 다음 조작은 조회 노드를 늘리는 것입니다.

이 이동은 아직 실측으로 확인하지 못했습니다. 2026-09-08 회차는 수요를 천장 아래로 고정했기 때문에 데이터베이스가 빨라져도 애플리케이션이 더 할 일이 없었습니다. 그 회차에서 실효 부하가 오르지 않은 원인은 7.6 이 적은 대로 부하 발생기입니다. 봇으로 걸면 수요가 발생기에 묶입니다.

남은 것은 그다음입니다. 발생기를 개방 루프로 바꾼 뒤에도 처리량에 천장이 있고, 그 천장이 어느 계층인지는 아직 특정하지 못했습니다. 후보 아홉을 소거했습니다. 판정 근거와 소거 이력은 제출_20260901/테스트_정본.md 6 절에 둡니다.

이것이 이 축의 남은 작업입니다.

7.9 백업과 복구

대상	방식	주기	상태
데이터베이스 물리 백업	블록 스토리지 스냅샷	일 1 회	목표. 미배선
데이터베이스 논리 백업	기저 백업 도구	주 1 회	목표. 미배선
미리 쓰기 기록 보관	오브젝트 스토리지로 전송	연속	목표. 미배선. 설정값이 자리표시자입니다
복제	대기 서버 1 대 비동기 스트리밍	24 시간	로컬 한 호스트 두 컨테이너로 확인. 클라우드 미배포. RPO 0 을 보장하지 않습니다 (ADR 0024)
실험 증거	판정 파일을 저장소에 커밋	실험마다	성립. harness/evidence/ 아래 회차 디렉터리 40 개와 결과 파일 7 개

백업 항목은 전부 목표입니다. 배선한 것으로 적지 않습니다.

7.10 침묵 판정

체결이 0 인 상태가 두 가지입니다. 정말 거래가 없는 것과, 무언가 고장 나서 주문이 안 들어오는 것입니다. 둘을 구분해야 합니다.

봇 플릿이 24 시간 기저 트래픽을 만드는 것이 그 구분 수단입니다. 봇이 도는데 체결이 0 이면 평온이 아니라 사고입니다.

다만 봇 하나에 판정을 걸지 않습니다. 봇은 내부 경로만 왕복하므로 로드밸런서와 콘텐츠 전송 네트워크 구간의 고장을 잡지 못합니다. 외부에서 찌르는 프로브가 그 구간을 맡아야 하는데 아직 배선하지 않았습니다.

8. 보안

8.1 접근 경로

대상	경로	통제
고객 평면	콘텐츠 전송 네트워크에서 로드밸런서	공개
관리 평면	별도 포트, 상호 인증서 검증	인증서 신원
하네스 API	상시면에 등록하지 않음	비활성

상태 확인 경로	내부에서만	외부 라우팅 없음
서버 셸	SSM 세션 매니저	원격 접속 포트 미개방

원격 접속 포트를 아예 열지 않습니다. 세션 기록이 그 자체로 감사 증거입니다.

8.2 파괴 액션 승인

마켓 정지, 노드 종료, 권한 변경 같은 되돌리기 어려운 조작은 승인 티켓을 요구합니다.

예외가 하나 있습니다. 카오스 주입과 해제 4 종은 티켓이 선택입니다 (ADR 0014). `chaos.kill_node`, `chaos.network`, `chaos.app`, `chaos.network.clear` 넷입니다. 토큰 없이 부르면 승인 없이 진행하고 승인 식별자를 빈 값으로 적재합니다. 토큰을 제시하면 종전 계약이 그대로 걸립니다. 완화 대상은 이 넷뿐이고 봇 파라미터, 마켓 정지와 재개, 운영자 생성과 역할, 회원 폐쇄는 필수로 남습니다.

곧 "노드 종료" 는 관리 평면의 노드 종료를 뜻하고 카오스 주입의 노드 정지는 이 예외에 듭니다.

승인은 1 인 승인입니다. 티켓을 발급한 운영자와 사용하는 운영자가 같아도 됩니다 (ADR 0019).

종전 판은 서로 달라야 한다고 걸었습니다. 그런데 이 프로젝트의 운영 인원은 1 인입니다. 두 인증서를 한 사람이 번갈아 제시하는 것은 담합 방지가 아니라 그 외형일 뿐이었고, 실제로 한 일은 야간 단독 운영 시간대에 정지 명령을 봉쇄한 것이었습니다.

바꾼 뒤에도 남는 방어가 일곱입니다.

- 승인 티켓을 따로 발급하는 단계. 실행 전에 의도를 한 번 더 적습니다
- 티켓 수명 5 분
- 한 번 쓰면 소각
- 액션과 대상에 묶음. 다른 대상에 전용할 수 없습니다
- 요청자 묶음. 남의 승인을 가로챌 수 없습니다
- 역할 통제
- 추가만 가능한 감사 기록

사라지는 것은 발급자와 사용자의 분리 한 겹입니다. 단독 운영자의 고의는 사전에 막히지 않고 감사 기록으로만 사후에 드러납니다. 이 대가를 한계에 등재했습니다.

8.3 가드레일

비채택으로 정한 서비스는 계정 권한에서 생성 자체를 막습니다. 문서로만 "안 쓴다"고 적으면 실수로 켜집니다.

9. 비용

9.1 원칙

- 단가는 클라우드 사업자의 공식 가격 파일을 원천으로 씁니다. 요약 사이트 수치를 쓰지 않습니다
- 원천에 없는 값은 유도값 또는 미확인으로 표시하고 그 사실을 표에 남깁니다
- 환산 환율은 1 달러 1,400 원입니다. 민감도를 함께 적습니다
- 자체 상한은 월 500,000 원입니다. 2026-09-08 지도교수 승인으로 종전 100,000 원에서 올렸습니다. 승인액은 50 만원 이상이고 크레딧으로 지급 예정이며 지급 확인은 아직입니다
- 부가세 10 퍼센트를 별도 계상합니다
- 상한은 달력 월 실청구로 판정합니다

9.2 상시면 단가

768 시간 기준입니다.

항목	산식	달러
로드밸런서 고정분	0.0225 x 768	17.2800
로드밸런서 변동분	0.008 x 1 x 768	6.1440
애플리케이션 t4g.small 2 대	0.0208 x 2 x 768	31.9488
데이터베이스 t4g.small 2 대	0.0208 x 2 x 768	31.9488
봇과 관측 t4g.small 1 대	0.0208 x 768	15.9744
주소 변환 게이트웨이 2 개	0.059 x 2 x 768	90.6240
주소 변환 데이터 처리 20 기가	20 x 0.059	1.1800
공인 주소 2 개	0.0050 x 2 x 768	7.6800
블록 스토리지 5 노드 20 기가	100 x 0.0912 x 768/730	9.5947
외부 전송 20 기가 추정	20 x 0.126 x 768/730	2.6512
도메인과 오브젝트 스토리지	정액	0.7200
확정 단가 소계		215.7459

가용영역 간 전송과 세션 매니저 요금은 확인하지 못해 소계에 넣지 않았습니다. 따라서 215.7459 는 확정 단가만의 하한입니다.

주소 변환 게이트웨이 단가 시간당 0.059 달러와 데이터 처리 기가당 0.059 달러는 서울 지역 공시가로 적었으나 청구서로 확인하지 못했습니다. 배포 후 실청구로 검증할 항목이며 13.5 에 등재했습니다. 나머지 단가는 종전 판이 쓰던 값 그대로입니다.

블록 스토리지가 8 노드에서 5 노드로 줄어든 것은 주소 변환 인스턴스 두 대가 사라지고 대기 서버 한 대를 줄였기 때문입니다 (ADR 0020, ADR 0024).

9.3 달력 월 실청구

730 시간 전량 가동 기준입니다. 상한 판정은 이 값으로 합니다.

항목	산식	달러
로드밸런서 고정분	0.0225 x 730	16.4250
로드밸런서 변동분	0.008 x 1 x 730	5.8400
애플리케이션 t4g.small 2 대	0.0208 x 2 x 730	30.3680
데이터베이스 t4g.small 2 대	0.0208 x 2 x 730	30.3680
봇과 관측 t4g.small 1 대	0.0208 x 730	15.1840
주소 변환 게이트웨이 2 개	0.059 x 2 x 730	86.1400
주소 변환 데이터 처리 20 기가	20 x 0.059	1.1800
공인 주소 2 개	0.0050 x 2 x 730	7.3000
블록 스토리지 5 노드 20 기가	100 x 0.0912	9.1200
외부 전송 20 기가 추정	20 x 0.126	2.5200
도메인과 오브젝트 스토리지	정액	0.7200
달력 월 소계		205.1650

달력 월	소계 달러	부가세 포함	원화 환산	상한 500,000 원
2026 년 9 월	205.17	225.68	315,954 원	통과. 여유 184,046 원
2026 년 10 월	205.17	225.68	315,954 원	통과. 여유 184,046 원

실제로는 9 월 청구가 이보다 적습니다. 아직 클라우드에 인스턴스를 세운 적이 0 건이고 달을 온전히 채우지 못하기 때문입니다. 전량 가동으로 잡은 것은 상한 판정을 보수적으로 하기 위해서입니다.

종전 판은 이 자리에 76.13 달러를 적었으나 그 값이 어떻게 나왔는지를 적지 않았습니다. 768 시간 소계에서도 730 시간 환산에서도 재현되지 않아 근거를 세울 수 없었습니다. 22 절이 수치에 측정 근거를 요구하므로 유도를 다시 세웠습니다.

9.4 환율 민감도

환율	달력 월 실청구	상한 대비
1,350 원	304,670 원	여유 39.1 퍼센트
1,400 원	315,954 원	여유 36.8 퍼센트
1,450 원	327,238 원	여유 34.6 퍼센트
1,500 원	338,522 원	여유 32.3 퍼센트

손익분기 환율은 2,216 원입니다. 관리형 게이트웨이로 바꾸면서 여유가 76 퍼센트대에서 30 퍼센트대로 줄었습니다. 그래도 환율이 상한 판정을 위협하지는 않습니다.

9.5 총계

항목	달러
상시면 확정 단가분	215.75
증명면 창 구동 90 시간	7.79
소계	223.54
부가세 10 퍼센트	22.35
총계	245.89

원화 환산 344,245 원입니다.

지원 범위 안입니다. 기존 크레딧 잔액이 163.71 달러이고, 여기에 2026-09-08 지도교수 승인분 50 만원 이상이 다음 주에 크레딧으로 지급될 예정입니다. 승인분만으로도 총계를 덮습니다. 지급 확인 전까지는 기존 잔액 163.71 달러 기준으로 82.18 달러가 부족한 상태이므로, 지급이 늦어지면 그 차액이 현금 지출이 됩니다. 주소 변환을 관리형으로 바꾸면서 13.89 달러에서 늘었고, 대기 서버를 한 대 줄이면서 다시 내려왔습니다 (ADR 0020, ADR 0024).

표의 행 합을 권위로 삼습니다. 종전 판들이 소계와 본문 인용값을 따로 관리하다 두 번 어긋났습니다.

10. 검증

10.1 판정 원칙

사람 눈으로 판정하지 않습니다. 판정은 셋 중 하나입니다.

- SQL 위반 행 수. 0 행이면 통과입니다
- 지표 임계. 넘으면 실패입니다
- 종료 코드. 0 이 아니면 실패입니다

실험 회차의 기록과 각 회차가 유효한지에 대한 판정은 `제출_20260901/테스트_정본.md` 가 갖습니다. 무효 회차와 그 사유, 판정이 뒤집힌 이력도 그 문서에 있습니다.

10.2 무결 판정

복식부기 원장의 불변식 27 개를 SQL 로 검사합니다. 자산별 합이 0 인지, 주문이 중복 접수되지 않았는지, 체결이 중복되지 않았는지, 명령이 유실되지 않았는지를 각각 다른 질의가 봅니다.

파일은 `harness/sql/invariants.sql` 입니다. 한 항목이라도 1 행을 돌려주면 그 회차는 실패입니다.

27 개는 그 파일의 판정 블록 수입니다(2026-09-09 실측). 코드 축은 C01 부터 C25 까지이고, 그중 둘이 각각 두 블록으로 갈라져 블록이 27 개가 됩니다. C07 은 주문 표와 저널을 따로 보아 C07 과 C07P 로, C13 은 구현 여부와 저널 기록 여부를 따로 보아 C13a 와 C13b 로 나뉩니다.

애플리케이션이 주기적으로 도는 검증기는 26 개입니다(`InvariantCheck` 열거형). 27 개에서 C04 하나가 빠집니다. C04 는 지연 제약 트리거가 데이터베이스 안에서 직접 강제하므로 따로 조회할 대상이 아닙니다. 곧 27 개가 카탈로그의 크기이고 26 개가 앱이 순회하는 크기이며, 둘의 차이는 강제 수단이 다른 한 건뿐입니다.

10.3 장애 주입 카탈로그

주입 스크립트 9 종입니다. 각 실험이 무엇을 어떻게 넣는지 적습니다.

실험	주입 대상	원시 연산
E1	데이터베이스 경로 지연	프록시 지연 주입
E2	데이터베이스 연결 절단	프록시 타임아웃 0
E3	캐시 계층 정지	컨테이너 강제 종료
E4	리더 노드 정지	컨테이너 강제 종료
E5	네트워크 분단	네트워크에서 컨테이너 분리
E6	폭주 중 페일오버	봇 부하 상향 후 강제 종료
E7	회색 장애	컨테이너 일시 정지
E8	애플리케이션 내부 교란	주입 도구
E9	커밋 경로 절단	프록시 연결 재설정
E10	커널 계층 지연	미실시. 검증면 전용입니다

E1 부터 E9 까지는 전부 프록시와 컨테이너 조작입니다. 커널 큐의 실제 거동을 재현하지 않습니다. 그것이 E10 의 몫이고 아직 돌리지 않았습니다.

10.4 실측 증거

저장소에 판정 산출물 30 건이 있습니다. 2026-09-08 회차가 가장 최근입니다.

증거	무엇을 쟀는가
`replication-fault-20260908`	동기 복제 대상 상실이 쓰기에 미치는 영향과 정족수의 효과
`sync-replication-20260908`	동기 복제 자체의 처리량 대가
`leader-lease-dynamodb-20260908`	임차를 커밋 경로 밖으로 옮긴 효과
`leader-failover-20260908`	두 노드 리더 교체 시간과 세대 번호
`follower-rejoin-20260908`	부하 중 노드 재합류
`pool-db-matrix-20260908-203140`	커넥션 풀과 데이터베이스 규격 교차. 40 OPS
`pool-db-matrix-20260908-204634`	같은 교차. 고부하 시도
`asg-scenario-20260908`	확장과 축소 소요 시간
`wal-fsync-20260907`	저장소 동기화 지연
`ws-admission-20260907`	동시 접속 상한

나머지 20 건은 용량 보정과 처리량 측정입니다.

10.5 주요 실측값

항목	값	출처
리더 교체 시간	9.6 초에서 13.1 초	`follower-rejoin-20260908`(9.6 초)과 `leader-failover-20260908`(11.3, 12.0, 13.1 초)
부하 중 재합류	20.8 초	`follower-rejoin-20260908`
동기 복제의 처리량 대가	복제 없음 76.3, 비동기 65.9, 동기 53.0 (초당)	`sync-replication-20260908`
대기 서버 1 대 상실 시 접수	초당 97.2 에서 0	`replication-fault-20260908`
정족수에서 1 대 상실 시 접수	초당 56.0 유지	같음
임차 분리 후 정지 중 갱신	초당 0.35 회 지속	`leader-lease-dynamodb-20260908`
데이터베이스 규격 상향의 지연 효과	상위 1 퍼센트 60 퍼센트 이상 감소	`pool-db-matrix-20260908-*`
확장 소요	컨테이너 생성 1.0 초, 앱 기동 15.8 초, 투입 1.2 초	`asg-scenario-20260908`
주문당 라이터 시간	10.1 밀리초. 그중 동기화 6.41 밀리초	`wal-fsync-20260907`

10.6 통합 검증

검증 클래스 74 개가 전량 통과합니다. 실제 데이터베이스를 띄워 도는 통합 검증이 포함됩니다.

지속적 통합 워크플로 4 종이 병합 전에 검증합니다. 백엔드 검증, 웹 검증, 이미지 빌드, 스타일 검사입니다.

10.7 무엇이 아직 검증되지 않았는가

- 배포 대상 인스턴스 실물에서 잰 값이 0 건입니다
- 커널 계층 주입이 미실시입니다
- 클라우드 환경에서의 가용영역 상실 재현이 미실시입니다
- 병목 이동(데이터베이스를 키운 뒤 애플리케이션이 병목이 되는 것)이 미확인입니다

11. 일정

11.1 남은 구간

구간	기간	내용
W1	09-09 부터 09-15	클라우드 계정과 네트워크 구성
W2	09-16 부터 09-22	노드 배포와 도메인 연결
W3	09-23 부터 09-29	데이터베이스 이중화와 권한 분리
W4	09-30 부터 10-06	관측 배선과 알람 실투입

W5	10-07 부터 10-09	커널 계층 주입과 병목 이동 실측
W6	10-10 부터 10-16	최종 완주와 발표 준비

10-09 에 기능을 동결합니다. 그 뒤에는 실험과 문서만 합니다.

11.2 잘라낼 순서

시간이 모자라면 아래 순서로 뺍니다.

1. 애플리케이션 내부 교란 실험
2. 프록시 지연 실험 재실행
3. 커밋 절단 실험 재실행

무결과 복구 판정에 직접 걸린 실험은 마지막까지 남깁니다.

12. 완료 기준

발표 시점에 아래가 전부 참이어야 합니다.

번호	기준	판정 방법
A1	외부에서 도메인으로 접속됩니다	브라우저 접속 확인
A2	불변식 27 개의 위반 행이 0 입니다	SQL 실행 결과
A3	리더 교체가 30 초 이내에 끝납니다	판정 파일의 시각 차이
A4	세대 번호가 정확히 1 증가하고 구 리더 쓰기가 0 입니다	판정 파일
A5	실험 판정 파일이 저장소에 남습니다	파일 존재 확인
B1	2026 년 9 월과 10 월 달력 월 실청구가 각각 500,000 원 이하입니다	청구서. 계획값 각 342,146 원
B2	장기 접근 키가 0 개입니다	계정 설정 확인
C1	병목 계층을 관측으로 특정하고 조작 효과를 실측합니다	조작 전후 지표 비교

C1 이 이번에 추가된 기준입니다. 현재 데이터베이스 축은 성립했고 애플리케이션 축이 미확인입니다.

13. 한계

정직하게 적습니다. 여기 적힌 것은 알고 있으면서 해결하지 못한 것입니다.

13.1 측정 평면

번호	한계
1	배포 대상 인스턴스 실물 실측이 0 건입니다. 로컬 도커 값이

	그대로 성립한다는 보장이 없습니다
2	같은 기계에서 상시 스택 12 개가 돌았고, 통합 검증이 겹친 구간에는 임시 데이터베이스 컨테이너가 최대 27 개 더 붙었습니다. 곧 27 은 총수가 아니라 덧붙은 수입니다. 성능 수치는 절대값이 아니라 주행 안의 상대 비교입니다
3	가용영역 간 지연을 프록시로 모사했습니다. 실제 지연은 미측정입니다
4	저장소 동기화 지연의 거의 전부가 개발 기계의 파일 시스템 계층에서 나옵니다. 배포 볼륨에서 다시 재야 합니다

13.2 구조

번호	한계
5	쓰기는 단일 라이터입니다. 노드를 늘려도 쓰기 천장이 오르지 않습니다
6	가용영역 a 를 잃으면 쓰기가 멈춥니다. primary 가 그 영역에 있고 대기 서버 승격이 수동입니다. 승격 명령은 1.7 초대이나 감지와 판단과 접속 대상 전환을 포함한 실제 복구 시간은 미측정입니다 (ADR 0024)
7	시계 스큐 내성이 검증 범위 밖입니다. 임차 만료 판정이 노드 시계에 걸립니다
8	봇과 관측이 한 노드에 있습니다. 그 노드가 죽으면 기저 트래픽과 지표 수집이 함께 멈춥니다
9	승격 재생이 엔진 상태를 두 벌로 만듭니다. 그 순간 메모리를 두 배로 씁니다
10	RPO 0 을 보장하지 않습니다. primary 급사 시 대기 서버에 닿지 않은 미리 쓰기 기록이 유실될 수 있고, `command_journal` 이 같은 데이터베이스에 있어 그 구간은 저널 재생으로도 복원되지 않습니다. 유실 창은 복제 지연만큼입니다. 2026-09-12 실측에서 복제가 따라오는 중 급사는 0 건, 대기 서버를 8 초 멈춘 뒤 급사는 응답을 받은 122 건 중 36 건이었습니다 (ADR 0024)

13.3 통제

번호	한계
11	단독 운영자의 고의를 사전에 막지 못합니다. 승인을 1 인으로 바꾼 대가입니다. 감사 기록으로만 사후에 드러납니다
12	자동 확장 정책이 없습니다. 사람이 자는 동안에는 증설이 일어나지 않습니다

13.4 미배선

번호	한계
13	상시면 컴포즈에 관측 서비스 정의가 0 건입니다. 관측은 로컬 카오스 스택에서만 돕니다
14	계층 프로브가 검증면에만 있습니다. 2026-09-09 에 진입 프로브와 데이터베이스 지표와 컨테이너 자원을 배선했고 알람 둘이 걸렸으나, 상시면 컴포즈에 관측 정의가 0 건이라(항목 13) 그 평면에서는 아직 돌지 않습니다. 프로브가 같은 사설망 안에 있어 콘텐츠 전송 네트워크 구간은 여전히 못 봅니다
15	알림 웹훅 값 파일이 없습니다. 수신 경로 자체는 배선돼 있습니다. `harness/alertmanager/alertmanager.yml` 이 receiver 를 chat 하나로 두고 discord_configs 의 webhook_url_file 로 `/etc/alertmanager/secrets/discord_webhook_url` 을 가리킵니다. 그 디렉터리에 있는 것은 예시 파일과 무시 규칙뿐이고 값 파일이 없습니다. 파일이 없어도 Alertmanager 는 기동합니다(2026-09-10 실측). 실패는 통지 시점에 납니다
16	백업 3 종이 전부 목표 상태입니다
17	인프라 코드화 파일이 0 개입니다

13.5 미확인

번호	한계
18	병목 이동이 미확인입니다. 데이터베이스를 키운 뒤 애플리케이션이 병목이 되는 것을 실측하지 못했습니다. 수요를 천장 아래로 고정했기 때문입니다
19	실효 부하 상한이 두 층입니다. 봇으로 재면 발생기가 상한이고 k6 로 재면 수용 게이트가 상한입니다. 2026-09-09 에 판정했습니다. 요청을 20 에서 400 으로 20 배 올렸는데 실효 처리량은 초당 7.0 에서 11.3 으로 1.6 배에 그쳤고 200 에서 13.3 으로 꺾였습니다. 같은 창에서 수용 거절 0, 커넥션 풀 대기 0, 시퀀서 큐 0, 봇 노드 CPU 4.8 에서 7.2 퍼센트, 애플리케이션 노드 CPU 4.0 퍼센트 이하였습니다. 서버 어느 계층도 포화하지 않았습니다. 원인은 봇 수요입니다. 활성 봇이 14 대이고 발주 주기가 고정 슬립이라 토큰버킷이 천장으로만 작동합니다. 실제 발행량이 봇 수요와 목표치 중 작은 쪽이므로 목표만 올리면 아무 일도 일어나지

	않습니다. 곧 종전의 40 회차와 200 회차는 같은 부하였습니다. 같은 날 개방 루프 계측기로 다시 재니 그림이 달랐습니다. 요청 초당 20 에서 이미 접수 15.8, 수용 거절 101.6, p99 971 밀리초, 시퀀서 큐 13 이었고, 요청을 올리면 접수가 오르지 않고 줄면서 거절만 늘어 초당 350 요청에서 거절이 303 에 이르렀습니다. 커넥션 풀 대기는 전 구간 0 이므로 커넥션 부족이 아니라 정원 밖 요청을 즉시 거절하는 설계가 동작한 것입니다. 봇은 목표를 400 으로 올려도 거절을 한 건도 만들지 못했고 시퀀서 큐도 0 이었습니다. 부하 발생기를 시스템 밖에 두는 설계의 근거가 이 대조입니다 (`harness/evidence/k6-ceiling-20260909/`)
20	대기 서버에 읽기를 보내면 커밋이 느려지는지 미측정입니다. 2026-09-09 에 실행을 시도했고 도구 결함 셋을 잡았습니다. 복제 접속을 막는 기본 접근 규칙, 윈도우에서 실행되지 않는 셸 호출, 부하 0 구간에서 분위수가 값을 내지 못하는 표본 창입니다. 앞의 둘은 고쳤고 세 번째는 부하를 세운 뒤 다시 돌려야 합니다. 종전 판이 "실험 도구는 준비했습니다"로 적었으나 그 도구는 한 번도 끝까지 돈 적이 없었습니다
21	버스트 가능 인스턴스의 신용 소진이 측정에 미치는 영향이 미확인입니다. 데이터베이스가 상시 0.4 에서 0.5 코어를 쓰는데 이 등급의 기저 성능이 그 근처입니다
22	리더 임차의 DynamoDB 경로가 기본값이 아닙니다. 2026-09-08 회차에서 그 경로를 재어 87 초 정지를 견디는 것을 확인했으나, 배포 기본값은 데이터베이스 쪽이라 상시 구동은 종전 경로로 돕니다. 상시면에서 어느 쪽을 켤지 정하지 않았습니다
23	주소 변환 게이트웨이 단가를 청구서로 확인하지 못했습니다. 시간당 0.059 달러와 기가당 0.059 달러는 공시가이고, 이 값이 9.2 와 9.3 계산 전체를 떠받칩니다. 배포 후 실청구로 검증합니다 (ADR 0020)
24	부하 발생기의 수요를 올리는 손잡이가 하네스에 없습니다. 격자 실험이 세팅하는 부하 변수가 목표 초당 처리량 하나뿐이고 봇 대수도 도착률도 건드리지 않습니다. 수요를 실제로 키우는 경로는 시나리오 프리셋인데 실험이 쓰지 않습니다
25	봇 노드 메모리가 상한의 71 퍼센트입니다. 봇 하나가 HTTP 클라이언트를 하나씩 들고 있어 360 대에 스레드 366 개와 파일 서술자 757 개를 씁니다. 봇 대수를 올리면 이 축이 먼저 걸립니다
26	수용 정원과 커넥션 풀의 최적값을 찾지 못했습니다. 2026-09-09 에 세 번 시도했고 세 번 다 무효였습니다. 첫 회차는 설정이 컨테이너에 닿지 않아 네 칸이 같은 조건이었고,

	둘째는 정원을 축으로 잡았으나 풀 고정에서 정원을 열세 배 벌려도 체결이 움직이지 않았으며, 셋째는 풀을 축으로 놓았으나 회차 도중 데이터가 쌓여 축 효과와 시간 효과가 섞였습니다. 회차 종료 시점 주문 61.7 만 건, 데이터베이스 2.15 기가바이트입니다. 남는 사실은 둘입니다. 풀 대기가 전 회차 0 이라 커넥션 부족이 아니고, 풀을 여덟 배 벌려도 체결이 초당 2.4 에서 4.0 사이에 머물러 풀이 정점을 만드는 축이 아닙니다 (harness/evidence/pool-ladder-20260909/)
27	부하 계정 키가 데이터베이스와 갈릴 수 있고 그 상태가 조용합니다. 2026-09-09 에 실제로 갈려 있어 계측기 주문이 전량 거부됐는데, 접수 지표가 봇 값으로 채워져 그럴듯하게 보였습니다. 상태 코드를 쪼개기 전에는 드러나지 않습니다. 회차 시작 전 키 일치 확인이 없습니다
28	회차 간 편차가 큽니다. 같은 설정에서 잰 p99 가 971 밀리초와 423 밀리초로 두 배 넘게 벌어졌습니다. 단일 표본으로 설정 효과를 판정하면 안 되는 규모이고, 실제로 초판 해석에서 다른 날 회차와 비교해 잘못 읽었습니다
29	처리량 상한은 코드가 커넥션을 하나씩만 쓰는 데 있습니다. 2026-09-10 에 특정했습니다. 부하 중 데이터베이스의 활성 연결을 141 회 표본하니 평균 0.40 개였고 표본의 73.8 퍼센트가 0 개였습니다. 풀에 32 개를 열어 두고 그렇습니다. 같은 창에서 수용 거절이 초당 100 건이었습니다. 초당 100 건을 거절하면서 데이터베이스가 놀고 있었습니다. 축 셋을 벌려도 접수가 움직이지 않는 이유가 이것입니다. 풀 16 에서 128 로 여덟 배에 접수 불변이고 풀 대기 0, 데이터베이스 코어 1 에서 8 로 여덟 배에 접수 6 퍼센트, 쓰기 정원 12 에서 48 로 네 배에 접수 불변에 지연 2.5 배 악화입니다. 데이터베이스는 어느 규격에서든 한 코어의 0.6 에서 0.94 만 씁니다. 늘린 자원을 코드가 쓰지 않으므로 무엇을 늘려도 소용이 없습니다. 고치는 것은 이 판에 담지 못합니다 (제출_20260901/테스트_정본.md 5.6) 2026-09-10 에 고치기 시작했습니다. 마켓별 라이터로 가르되 그 앞에 채번 경합을 먼저 풉니다. event_seq 채번이 journal_counters 한 행의 UPDATE 이고 그 잠금이 커밋까지 유지되며 원장 게시보다 앞에 있어 정산 전 구간을 직렬화합니다. 격리 실측에서 라이터 12 개에 전역 카운터가 초당 43.5, 마켓별 카운터가 514.5 였습니다. 1 에서 5 단계를 착지했고 실측으로 확인했습니다. 접수 천장이 초당 72.2 에서 101.6 으로 41 퍼센트 올랐고, 평균 활성 커넥션이 0.40 개에서 2.60 개로 6.5 배가 됐으며, 데이터베이스가 쓰는 코어가 0.76 에서 2.24 로 늘었습니다. 예고값 514.5 에 못 미치는 이유도

	특정했습니다. 잠금 경합이 아니라 명령 하나가 데이터베이스를 열아홉 번 왕복하는 것 자체가 상한입니다. 행 잠금 대기가 3.8 퍼센트뿐이고 나머지는 데이터베이스가 실제로 일하는 시간입니다. 직렬 점유가 1.927 로 1.0 을 넘어 라이터 여럿이 동시에 도는 것도 확인했습니다 (ADR 0021) **2026-09-11 에 상한을 끝까지 특정했습니다(5.7.9).** 마켓별 샤딩 뒤 남은 상한은 `EngineBridge.submitLock` 입니다. 모든 마켓의 모든 접수가 전역 락 하나를 지나고 그 안에서 저널을 기록하고 커밋합니다. 부하 구간에서 그 락의 보유 점유가 0.993 이고 전역 락 하나의 물리 상한이 1.0 입니다. 보유 1 회가 6.935 밀리초이고 그중 6.879 밀리초가 저널 기록입니다. 호출 수도 35,647 로 같습니다. 수용 정원을 12 에서 48 로 네 배 벌려도 접수가 132.49 에서 128.11 로 평평하고 대기 점유만 19.0 에서 43.0 으로 늡니다. 같은 창에서 라이터 점유는 상한 12.0 중 2.2 이고 데이터베이스는 4 코어 중 1.5 코어입니다. 곧 후보가 전부 소거되고 이 락 하나가 남았습니다. 락을 마켓별로 가르면 상한이 마켓 수만큼 오를 것으로 보이나 전역 `command_seq` 채번과 C05 결번 없음 불변식이 전역 축에 걸려 있어 아직 재지 않았습니다
30	실험 지표 라벨을 잘못 읽은 이력이 있습니다. `exchange_order_latency_seconds_count` 를 접수로 썼으나 그 타이머는 체결된 주문만 셉니다. 러너 넷과 증거 문서 넷을 정정했습니다. 상대 비교와 판정은 같은 지표끼리라 유효했으나 절대값을 접수 천장으로 읽으면 안 됩니다
31	설정 변경 실험에서 설정이 실제로 적용됐는지 확인하지 않은 이력이 있습니다. 정원 격자가 compose 의 환경변수 전달 규칙 때문에 네 칸을 같은 설정으로 돌고 잘못된 판정을 냈습니다. 러너마다 시작 전에 대상 설정값을 지표로 읽어 기대값과 대조하는 절차가 없습니다
32	측정 조건이 회차 사이에 변합니다. 데이터가 계속 쌓이므로 오전 회차와 오후 회차를 직접 비교할 수 없는데 오늘 여러 판정을 그런 비교 위에 세웠습니다. 사다리처럼 여러 칸을 순서대로 도는 실험은 뒤 칸일수록 데이터가 많아 축의 효과와 시간의 효과가 같은 방향으로 섞입니다. 칸마다 초기화하거나 순서를 섞는 절차가 없습니다
33	실험이 남긴 데이터가 측정을 오염시킵니다. 2026-09-09 하루 부하로 데이터베이스가 2,361 메가바이트가 됐고 열린 주문이 19.9 만 건 쌓였습니다. 볼륨을 파기하고 같은 부하를 다시 걸어 보니 체결이 초당 2.7 에서 14.0 으로 세 배 넘게, 읽기 대 쓰기 비율이 787 대 1 에서 116 대 1 로 바뀌었습니다. 곧 그날 회차 대부분이 그 오염 위에 섰습니다.

	회차마다 초기화하거나 데이터 상태를 함께 기록하는 절차가 없습니다 (harness/evidence/clean-baseline-20260909/)
34	6.4 의 조회 분산이 검증면에서 한 번도 돌지 않았습니다. 대상 그룹을 쓰기와 조회로 가르는 것은 상시면 설계이고 그 배선은 deploy/nginx-edge.conf.template 에 있습니다. 검증면 로드밸런서는 harness/nginx/nginx.conf 이고 위치 블록 하나에 상류 하나입니다. chaos-api-2 가 backup 이라 1 차가 죽어야 트래픽을 받습니다. 곧 평시에 조회조차 받지 않습니다. 그래서 노드를 늘렸을 때 무엇이 갈리는지를 이 평면에서는 잰 적이 없습니다. 증거 전부가 이 평면에서 나왔으므로 조회 분산의 실측값이 0 건입니다. 상시면에 인스턴스를 세워야 재집니다
35	발생기가 죽어도 표가 조용한 시스템으로 보입니다. http_server_requests 에 수용 거절과 인증 실패가 잡히지 않습니다. 둘 다 Micrometer 필터보다 앞에서 끊기기 때문입니다. 2026-09-09 정리된 사다리 열두 칸 중 여섯 칸이 이 결함으로 무효였습니다. 여섯 칸 모두 초당 16.7 이라는 같은 값을 냈고 그것이 봇 기저였습니다. 지연이 80 밀리초로 내려가고 데이터베이스 CPU 가 23 퍼센트였던 것이 단서였습니다. 포화라면 지연이 올라야 합니다. 러너에 종료 코드 확인과 제공 부하 하한 검사를 넣어 막았으나, 지표 자체가 제공 부하를 세지 않는다는 사실은 그대로입니다 (제출_20260901/테스트_정본.md 3.2)
36	매칭 라이터가 전역 하나입니다. 마켓 단위로 가르지 않았습니다. 단일 라이터를 둔 사유는 재생 결정론입니다. 저널을 처음부터 재생해 같은 상태가 나와야 하고 그러려면 전역 단일 순서가 필요합니다(불변식 C17). 그러나 그 사유가 요구하는 것은 마켓 안의 단일 순서이지 거래소 전체의 단일 순서가 아닙니다. 마켓마다 라이터를 하나씩 두면 순서 보장을 유지하면서 마켓 수만큼 수평 확장됩니다. 지금은 KRW-BTC 와 KRW-ETH 가 같은 라이터를 씁니다. 2026-09-10 지도 상담에서 지적받았습니다 2026-09-10 에 착수했습니다. 순서 축을 전역에서 마켓 안으로 옮기는 1 에서 4 단계가 착지했습니다. market_seq 를 새로 두고 event_seq 를 그 파생으로 바꿔 전역 채번을 끊었으며, 재생 커서를 (source_cmd_seq, event_ordinal) 축으로 옮겼습니다. 마켓별 재생이 전역 재생과 같은 북을 내는 것을 시험으로 확인했습니다. 라이터를 실제로 가르는 5 단계도 착지했습니다. 마켓마다 루프와 엔진을 하나씩 두고 접수를 마켓으로 라우팅합니다. 인스턴스는 늘리지 않았습니다. 남은 것은 되돌릴 수 없는 정리인 6 과 7 단계입니다 (ADR 0021)
37	리전 상실을 대비하지 않았습니다. 13.2 가 가용영역 상실은

	등재했으나 리전 단위 상실은 설계에 없습니다. 주 데이터베이스와 대기 데이터베이스가 같은 리전에 있습니다. 2026-09-10 지도 상담에서 지적받았습니다
38	수용 거절이 클라이언트 과다 요청과 서버 용량 부족을 같은 429 로 돌려보냅니다. 받는 쪽에서 자기 잘못인지 서버 잘못인지 갈리지 않고, 장애 때 원인 계층을 좁히지 못합니다. 실제로 2026-09-10 실측에서 초당 100 건을 429 로 돌려보내는 동안 포화한 자원이 하나도 없었습니다. 풀 대기 0, 활성 연결 0.40 개, 데이터베이스 CPU 0.6 코어였습니다. 곧 그 429 는 사실이 아니었습니다. 클라이언트 한도 초과는 429 로, 서버 용량 부족은 503 과 Retry-After 로 갈라야 합니다. 2026-09-10 지도 상담에서 지적받았습니다
39	데이터베이스 구성을 데이터베이스 전공 지도 상담 뒤에 교체할 예정입니다. 현재는 주 데이터베이스 한 대에 모든 SQL 을 보내고 대기 데이터베이스는 조회를 받지 않습니다. 2026-09-10 상담에서 이 구성 자체를 다시 보라는 지적을 받았고, 대안 넷을 받았습니다. 논리적 분할, 대기 서버 조회 분산, 미들웨어 계층, 수직 상향입니다. 이 중 수직 상향은 같은 날 실측으로 효과가 없음을 확인했고 그 사유가 한계 29 입니다
40	계정 없는 주문이 접수를 통과해 정산에서 노드를 내립니다. 2026-09-10 실측입니다. 부하 계정이 가지고 있지 않은 자산의 매도 주문을 내면 접수는 성공 응답을 주고, 그 뒤 라이터가 정산에서 계정 없음 예외를 만나 fail-fast 로 멈추며 리더십까지 함께 내려놓습니다. 곧 한 사용자의 잘못된 주문 하나가 노드를 통째로 내릴 수 있습니다. 접수 단계에서 자산 계정 존재를 확인하지 않는 것이 원인입니다 (5.7.3) 2026-09-10 에 고쳤습니다. 접수 경로의 가용 잔고 조회가 계정이 없으면 잔고 0 으로 보고 그대로 진행하던 것을 400 거부로 바꿨습니다. 잔고 0 과 계정 부재는 다른 사실입니다. 매수는 예약액이 금액이라 잔고 0 이면 가드에 걸리지만 매도는 예약액이 수량이라 그 자산 잔고를 보지 않고 통과합니다. 판정을 user_balances 로 해도 되는 근거는 그 뷰가 accounts 와 balances 의 조인이고 잔고 행을 계정 생성 트리거가 물리 강제하기 때문입니다. 실측으로도 사용자 계정 1,570 개 전부가 잔고 행을 가지고 있습니다
41	설계 문서가 데이터베이스 함수 본문을 통째로 옮겨 적어 두고 두 번의 개정 동안 갱신되지 않았습니다. `docs/contracts/백엔드_DB_설계.md` 가 `post_ledger_transaction` 의 V1 본문을 싣고 있었는데 그 함수는 V13 과 V27 에서 개정됐습니다. 2026-09-10 에 그 낡은 사본을 기준으로 새 판을 쓰다가 V27 이 없앤 재조회 세 건이 되살아나고 V13 이 넣은 원가 프로젝션 호출이

	사라졌습니다. 시험 열두 건이 깨져서 발견했습니다. 곧 문서가 코드와 갈린 것을 사람이 아니라 시험이 잡았습니다. 같은 날 그 자리를 마이그레이션 파일을 가리키는 참조로 바꿨습니다. 값을 두 곳에 두지 않는 것이 유일한 해법입니다. 같은 날 나머지도 훑었습니다. 그 문서가 옮겨 적은 함수 본문은 열일곱 개이고 그중 V1 이후 개정된 것이 둘입니다. `post_ledger_transaction` 과 `assert_txn_sealed_balanced` 이고 둘 다 낡은 사본이었습니다. 둘 다 참조로 바꿨습니다. 남은 열다섯 중 둘은 `prune_outbox_partitions` 와 `check_default_partitions` 로 마이그레이션에 정본이 아예 없는 로드맵 설계라 대조 대상이 아닙니다. 곧 사본과 정본이 같다고 말할 수 있는 것은 열셋입니다. 그 열셋도 같은 구조라 언제든 갈립니다
42	12 마켓 부하에서 주문의 3 분의 1 이 201 도 429 도 아닌 응답으로 빠졌습니다. 2026-09-10 실측입니다. 요청 240/s 칸에서 전체 주문 요청 171.5/s 중 접수 112.9, 수용 거절 87.1, 비 201 응답이 58.6/s 였습니다. 2026-09-11 에 사유를 갈랐습니다. 비 201 이 초당 60.93 이고 `exchange.orders.rejected` 의 `insufficient_funds` 가 정확히 같은 60.93 입니다. `domain_rejected` 는 0 입니다. 곧 400 이 전부 잔고 부족입니다. 부하 계정 잔고를 직접 보니 기준자산 열셋 중 아홉이 최소 0 이었습니다. 시드가 기준자산마다 100 인데 `order_flow.js` 의 매도 수량이 KRW-DOGE 와 KRW-TRX 에서 한 건에 50 이라 계정당 두 건이면 바닥납니다. 수량이 작은 KRW-BTC 0.001 과 KRW-ETH 0.01 만 99.84 와 98.33 으로 남았습니다. 같은 날 `clean_suite.py` 의 재발급이 기준자산 10^6 을 넣도록 고쳤습니다. 근거는 240/s 130 초에서 계정당 매도가 약 271 건이고 최대 수량 50 을 곱하면 13,550 이 필요하다는 계산입니다. 이 왜곡은 5.7.4 에서 5.7.7 까지의 12 마켓 수치 전부에 들어 있습니다. 각 회차의 전후 대조는 양쪽 조건이 같아 유효하지만 절대값은 시스템 천장이 아니라 시드가 만든 천장을 함께 봅니다. **해소(2026-09-11).** 고친 시드로 같은 사다리를 다시 걸었고 비 201 이 전 칸에서 0 입니다(5.7.8). 접수는 전 칸에서 올랐고 240/s 칸에서 113.25 가 125.71 이 됐습니다. 5.7.4 에서 5.7.7 까지의 절대값은 그대로 두되 이 한계를 함께 읽어야 합니다. 다시 재지 않는 이유는 그 회차들의 판정이 전부 전후 대조이고 대조는 왜곡에 영향받지 않기 때문입니다
43	마켓 순서 대역이 찰 때까지가 연 단위가 아니라 일 단위입니다. `event_seq` 를 `marketId * 10^7 + market_seq` 로 파생하므로 마켓당 지역 번호가 10^7 을

	넘으면 그 마켓의 채번이 예외를 던집니다. 2026-09-11 실측 기준으로 명령당 이벤트가 1.133 개이고 라이터 적용이 초당 118.73 건입니다. 12 마켓 균등이면 마켓당 초당 11.21 개이고 10^7 까지 10.3 일입니다. 봇이 몰리는 KRW-BTC 는 6.9 일, 단일 마켓 회차는 1.4 일입니다. 부하를 끄고 상시면만 돌려도 55 일입니다. 발표까지 남은 기간과 같은 자릿수입니다. 실패 양상이 나쁩니다. 채번 예외가 라이터를 fail-fast 로 멈추고 `WriterRouter.isRunning` 이 전체 기준이라 한 마켓의 정지가 전 마켓 접수 거부가 되며 리더십까지 내려놓습니다. 그 명령은 이미 저널에 커밋돼 있으므로 재기동해도 같은 자리에서 다시 죽습니다. 운영이 카운터를 되돌릴 수도 없습니다. `event_journal` 기본키와 두 레지스트리의 유일 제약이 `event_seq` 재사용을 물리로 막습니다. 지금은 디스크가 먼저 찹니다. 상시면 9 일, 부하시험 9 시간입니다. 그러나 보존 정책이 들어오는 순간 디스크 벽이 사라지고 대역이 다음 벽이 됩니다. 카운터 여유를 보는 경보가 없는 것도 함께 고칩니다. 불변식 C23 은 카운터와 저널의 정합만 보고 잔여 여유는 보지 않습니다
44	마켓 식별자가 20 이상이면 이벤트가 첫 건부터 `event_journal` 의 기본 파티션으로 갑니다. 대역 폭 10^7 과 p0 경계 2 x 10^8 이 정확히 20 배라 식별자 0 에서 19 까지만 p0 안에 듭니다. 오류가 아니라 무음 강등입니다. 그리고 기본 파티션에 행이 한 번 들어가면 2 x 10^8 부터의 파티션을 나중에 만들 수 없습니다. PostgreSQL 이 기본 파티션을 훑어 거부하기 때문입니다. V1 이후 `event_journal` 파티션을 더하는 마이그레이션이 하나도 없으므로 그 시점에 성장 경로가 영구히 닫힙니다. 현재 마켓이 열둘이라 도달하지 않았습니다. 불변식 C15 가 기본 파티션 유입을 WARN 으로 보고 있어 발견은 됩니다
45	스냅샷이 한 건도 찍히지 않습니다. 2026-09-11 실측입니다. 명령 121,812 건을 기록한 데이터베이스에서 `snapshots` 표가 0 행입니다. 원인은 배선입니다. `EngineConfig.snapshotCapture` 가 `setSnapshotHook` 을 레거시 `singleWriterLoop` 빈에만 겁니다. ADR 0021 5 단계가 운영 경로를 `WriterRouter` 샤드로 옮겼는데 그 샤드에는 훅이 없습니다. 저장소 전체에서 `setSnapshotHook` 호출부가 그 한 줄뿐입니다. 결과로 복구가 항상 명령 0 번부터 전량 재생입니다. 곧 복구 시간이 저널 크기에 비례합니다. 정본의 RTO 실측 0.107 초에서 14.367 초는 저널이 작던 회차의 값이라 이 배선에서는 저널이 자랄수록 함께 자랍니다. 아직 큰 저널로 페일오버를 재지 않았습니다
46	`command_journal` 의 파티션이 셋뿐이고 V1 이후 더한 마이그레이션이 없습니다. `p0` 가 10^8 까지, `p100000000` 이

	2 x 10^8 까지, 그다음이 기본 파티션입니다. 기본 파티션에 행이 한 번 들어가면 그 구간의 파티션을 다시 만들 수 없습니다. PostgreSQL 이 기본 파티션을 훑어 거부하기 때문입니다. 불변식 C15 가 유입을 보지만 WARN 이라 멈추지 않습니다. 부하시험 속도 초당 132 로 연속 주행하면 2 x 10^8 까지 약 17.4 일이고 봇만 도는 상시면 속도로는 1 년이 넘습니다. 하네스는 칸마다 볼륨을 파기하므로 도달하지 않습니다. 곧 급하지는 않으나 파티션을 미리 만들어 두는 비용이 거의 0 이라 다음 마이그레이션에 함께 넣습니다
47	처리량 천장의 절대값을 회차 밖으로 인용할 수 없습니다. 2026-09-12 실측입니다. 거의 같은 코드로 같은 스크립트에 같은 요청 240/s 를 걸었는데 접수가 157.47 에서 255.36/s 로 1.62 배 갈렸습니다. 제시된 부하는 양쪽 다 초당 256 에서 259 로 거의 같습니다. 주행 간 산포도 아닙니다. 같은 날 240/s 를 한 번 더 걸어 접수 255.16/s 를 얻었고 첫 주행 255.36 과 0.08 퍼센트 안에서 맞습니다. **그 차이를 아직 설명하지 못합니다.** 코드는 원인이 아닙니다. 두 회차가 같은 커밋은 아니지만 그 사이 서버 변경은 `EngineConfig.java` 6 행과 `CommandJournalAppender.java` 62 행 추가 2 행 삭제이고 내용이 `System.nanoTime` 표식과 `LongAdder` 뿐입니다. 계측은 비용을 더하는 방향이므로 더 많은 일을 하는 쪽이 더 빨랐습니다. 저장 계층은 실제로 움직입니다. `pg_test_fsync` 의 `fdatasync` 가 2026-09-07 건당 3.206 밀리초에서 2026-09-12 0.419 밀리초로 7.7 배 빨라졌고, `PGDATA` 를 직접 지정한 값과 기본 경로 값이 0.419 와 0.420 으로 같아 측정 위치 차이가 아닙니다. **다만 2026-09-11 의 저장 계층은 측정하지 않았습니다.** 그래서 이번 차이의 원인을 저장 계층으로 확정하지 않습니다. 3.206 밀리초를 그날에 대입하는 것도 안 됩니다. 같은 날 `stage0.json` 이 단일 세션 749.8 커밋/s, 평균 1.334 밀리초를 쟀는데 장치 플러시가 3.206 밀리초였다면 단일 세션이 초당 312 를 넘을 수 없습니다. 곧 09-11 의 플러시는 1.334 밀리초 이하였다는 상한만 알 뿐입니다. 그 상한도 무부하 창의 값입니다. `stage0` 은 데이터베이스 단독 기동이고 대조군은 부하 주행입니다. 모형 자체는 무너지지 않습니다. 락 보유 1 회의 역수가 단일 락의 산술 상한인데 09-11 은 6.166 밀리초로 초당 162 에 실측 157.47 이고, 09-12 는 2.420 밀리초로 초당 413 에 실측 373.82 입니다(요청 600/s 칸). 두 회차 모두 맞으므로 상한이 `EngineBridge.submitLock` 이라는 판정은 유지됩니다. 움직인 것은 그 락 안의 커밋 시간입니다. **따라서 72.2 에서 160.25 까지의 사다리는 변화율만 인용합니다.** 각 계단이 한

	회차 안의 전후 대조라 대조는 유효하지만, 설명되지 않는 변동이 1.62 배 크기로 있는 한 절대값을 날짜 너머로 시스템의 천장이라고 부르면 안 됩니다. 고치는 방향은 회차마다 저장 계층을 함께 재는 것입니다. 부하 직전 `pg_test_fsync` 한 번을 왕복 프로브와 사다리 주행 둘 다에 배선했고 이후 산출물이 그 값을 함께 담습니다 (`harness/evidence/stage0c-20260912/`, `제출_20260901/테스트_정본.md` 5.7.13)

13.6 발표 전에 반드시 닫아야 하는 것

13.4 의 미배선 다섯 항목 중 관측 배선과 알림 실투입은 완료 기준에 직접 걸립니다. 나머지는 한계로 남긴 채 발표합니다.

부록. 결정 기록

이 문서는 현재 상태의 정본입니다. "그때 왜 그렇게 정했는가"는 `docs/adr/`가 갖습니다. 결정 기록 21 편이 있습니다.

이 문서와 결정 기록이 어긋나면 이 문서가 현재 상태의 권위입니다. 다만 결정 기록의 사유를 뒤집으려면 새 결정 기록을 써야 합니다.